Università degli Studi di Siena

Project title: Performance Analysis of SpMV using CUDA, OpenMP and Advanced Formats (JDS, Hybrid)

Students: Giovanni Paolo Maugeri (matr. 165194)
Valentino Basili (matr. 164924)

Professor: Roberto Giorgi

Academic Year 2025/2026

Contents

1 Introduction

1.1 Overview

The objective of this project is to analyze and compare the performance of the Sparse Matrix-Vector Multiplication (SpMV) kernel across different architectures and storage formats. SpMV is a fundamental building block in many scientific computing applications, defined as $y = Ax$, where A is a sparse matrix and x, y are dense vectors.

Since the operation is strictly memory-bound, the choice of the storage format significantly impacts the effective bandwidth utilization. We implemented and tested the algorithm using **CUDA** (for GPU acceleration), **OpenMP** (for CPU parallelism), and a sequential C++ baseline.

We analyzed four main storage formats, specifically chosen to handle different sparsity patterns:

1. **CSR** (Compressed Sparse Row) - The standard baseline.
2. **ELL** (ELLPACK) - Optimized for structured grids on GPU.
3. **Hybrid** (ELL + COO) - A combination to handle moderate irregularity.
4. **JDS** (Jagged Diagonal Storage) - A technique to minimize warp divergence on GPUs.

2 Sparse Matrix Formats

2.1 CSR (Compressed Sparse Row)

The CSR format utilizes three arrays to compress the row indices, making it the standard format for sparse linear algebra on CPUs.

- **Arrays & Structure:**
  - `Value`: An array of size NNZ (number of non-zeros) storing the non-zero elements contiguously.
  - `Column`: An array of size NNZ mapping each element in `Value` to its column index.
  - `RowPtrs`: An array of size $N + 1$ (where N is the number of rows). It stores the starting index in `Value` for each row. The last element points to the end of the array (equal to NNZ).
- **Pros:** Efficient memory usage ($2 \cdot NNZ + N + 1$ storage); very fast for Matrix-Vector Multiplication (SpMV) on CPUs.
- **Cons:** Difficult to modify (inserting elements requires shifting arrays); on GPUs, it can cause load imbalance if row lengths vary significantly (some threads work much longer than others).

2.2 ELL (ELLPACK)

ELL fits the sparse matrix into a dense $N \times K$ structure, where K is the maximum number of non-zeros in any single row. Rows with fewer than K elements are padded with a sentinel value (usually zero).

- **Arrays & Structure:**
  - `Value`: A dense 2D array of dimensions $N \times K$.
  - `Index`: A dense 2D array of dimensions $N \times K$ storing the column indices. Padded entries usually store a dummy index.
- **Pros:** Perfect memory coalescing on GPUs (if Column-Major); removes the need for row pointers, allowing uniform control flow.
- **Cons:** Extremely inefficient for irregular matrices. If one row is very long ($max\_nnz \gg avg\_nnz$), the format allocates massive amounts of memory for padding zeros.

2.3 Hybrid (ELL + COO)

To mitigate the memory waste of ELL, the Hybrid format splits the matrix into two parts: a regular ELL portion and an overflow COO portion.

- **Arrays & Structure:**
  - **ELL Part:** Uses standard ELL arrays (`Value`, `Index`) of size $N \times K_{cutoff}$, where K_{cutoff} covers the majority of rows.
  - **COO Part:** Uses three arrays (`Row`, `Col`, `Val`) to store the "overflow" elements for rows longer than K_{cutoff}.
- **Pros:** Balances the high bandwidth of ELL with the flexibility of COO; effectively handles "power-law" matrices (many short rows, few very long rows).
- **Cons:** Higher implementation complexity (requires separate kernels or logic for each part); performance overhead when merging results from the two partitions.

2.4 JDS (Jagged Diagonal Storage)

JDS is explicitly designed to solve the Warp Divergence problem on GPUs by reordering the matrix rows.

- **Arrays & Structure:**
  - `Permutation`: An array of size N storing the mapping of original row indices to the sorted row indices.
  - `JD_Ptr`: An array pointing to the start of each "jagged diagonal" (iteration).
  - `Value` & `Col`: Arrays storing the non-zeros and column indices, ordered first by the jagged diagonal, then by the sorted row.

- **Pros:** Minimizes warp divergence on GPUs (threads in a warp process rows of similar length); ensures balanced workload.
- **Cons:** High preprocessing cost (sorting rows); requires gathering results back to original row positions (indirect memory access via the permutation array).

3 Implementation Details

3.1 Data Layout: Row-Major vs Column-Major

A critical optimization identified during development is the memory layout dependency on the architecture:

- CPU (OpenMP/Seq): Requires Row-Major layout for ELL/JDS to exploit cache locality.
- GPU (CUDA): Requires Column-Major layout (or coalesced access patterns) to maximize global memory bandwidth.

To have a fair comparison between CPU and GPU systems.

3.2 CUDA Kernels

We implemented four kernels:

- **CSR:** Standard scalar kernel (one thread per row).
- **ELL:** Coalesced kernel using Column-Major access.
- **Hybrid:** A two-step process where the ELL kernel initializes the result vector y, and a subsequent COO kernel accumulates the overflow using `atomicAdd`.
- **JDS:** Uses a permutation array `row_perm` to map results back to their original position. Threads iterate over the jagged diagonals. Since rows are sorted by length, divergence is minimal.

4 Experimental Results

4.1 Methodology

To evaluate the robustness of our implementations, we selected 5 matrices from the SuiteSparse Matrix Collection. These matrices were chosen to represent specific challenges for SpMV kernels.

Matrix	**Rows (N)**	**NNZ**	**Type**	**Challenge / Goal**
`cant`	62,451	4.0M	FEM	Good Baseline (Balanced)
`memsplus`	17,758	99K	Circuit	Irregular rows (Hybrid test)
`olafu`	16,146	1.0M	Structural	Dense rows, Structured
`scircuit`	170,998	959K	Circuit	High sparsity, Cache stress
`web-Google`	916,428	5.1M	Web Graph	Power-Law (JDS test)

Table 1: Benchmark Dataset from SuiteSparse Matrix Collection

4.2 Performance Analysis

4.2.1 Balanced and Structured Matrices: `cant` and `olafu`

Matrices derived from Finite Element Methods (FEM) like `cant` and `olafu` are the ideal scenario for structured formats. The ELL format dominates performance on CUDA.

- For `olafu` , ELL achieves a frame time of 0.06ms, significantly faster than CSR (0.12 ms). This confirms that when rows are uniform, the overhead of reading 'row_ptr' in CSR is higher than reading padded zeros in ELL.
- JDS also performs exceptionally well (0.05 ms for `olafu`), proving that sorting rows doesn't hurt performance on structured grids.

0.48

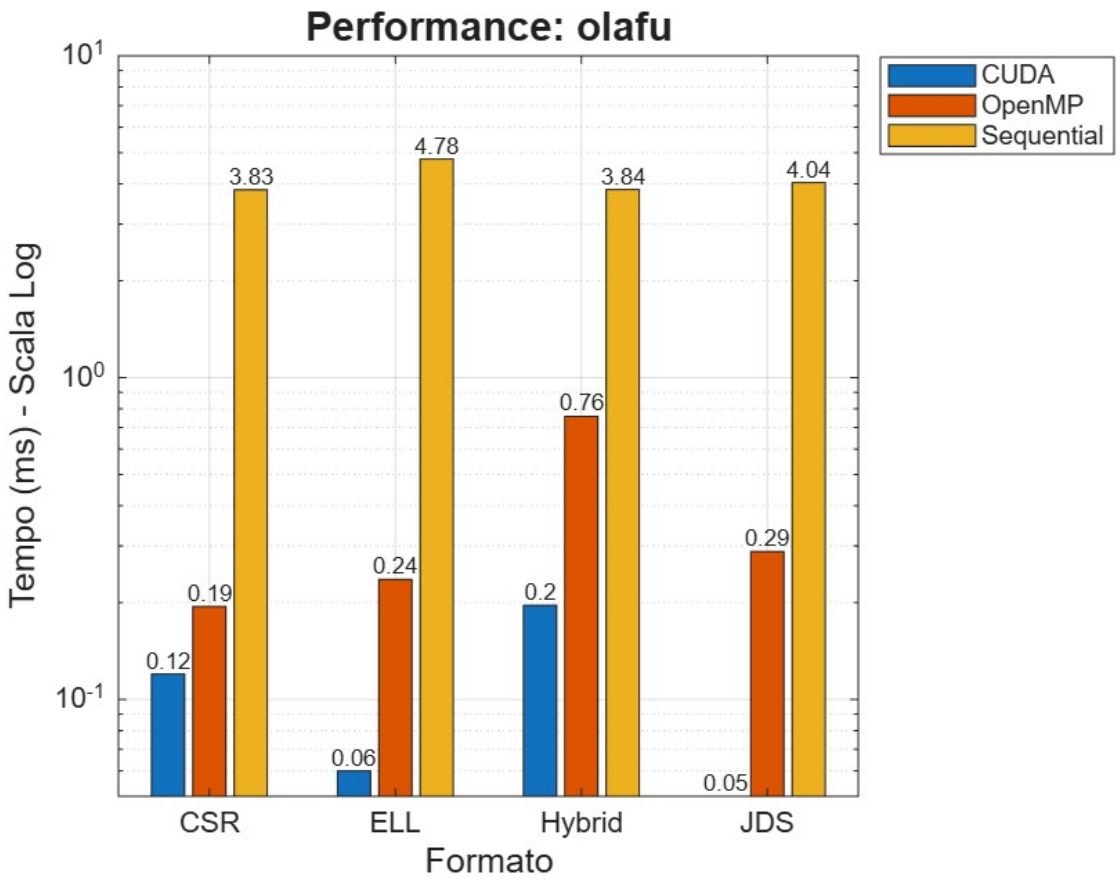

Figure 1: Matrix: `olafu`

0.48

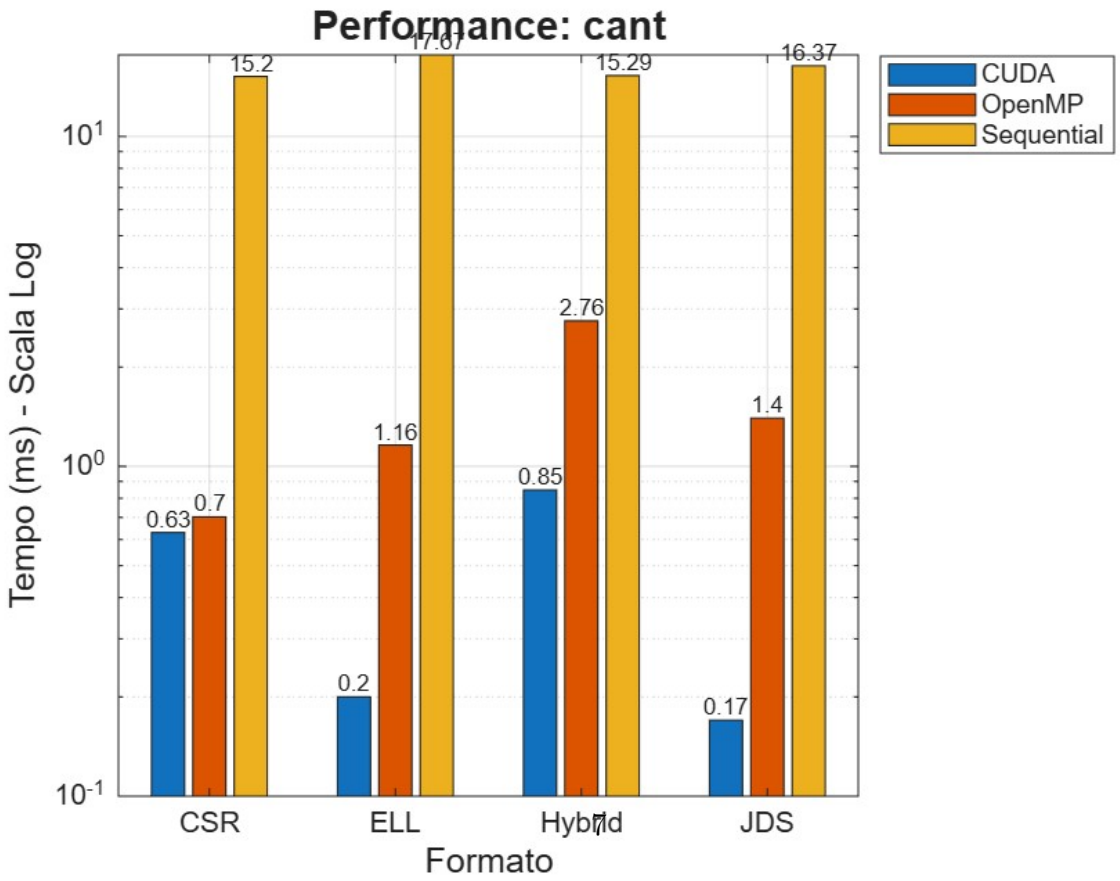

Figure 2: Matrix: `cant`

4.2.2 Irregular and Sparse Matrices: `memsplus` and `scircuit`

`memsplus` is a critical test case. It has a low average degree but high variance.

- The ELL Penalty: Notice the ELL bar on `scircuit` and `memsplus`. It is orders of magnitude slower compared to CSR. On `scircuit`, ELL takes 2.73 ms vs 0.07 ms for CSR. This visualizes the memory bandwidth wasted on padding.
- Hybrid Efficiency: The Hybrid format (0.02 ms on `memsplus`) matches the speed of CSR, proving it successfully handles irregularity without the memory penalty of ELL.

0.48

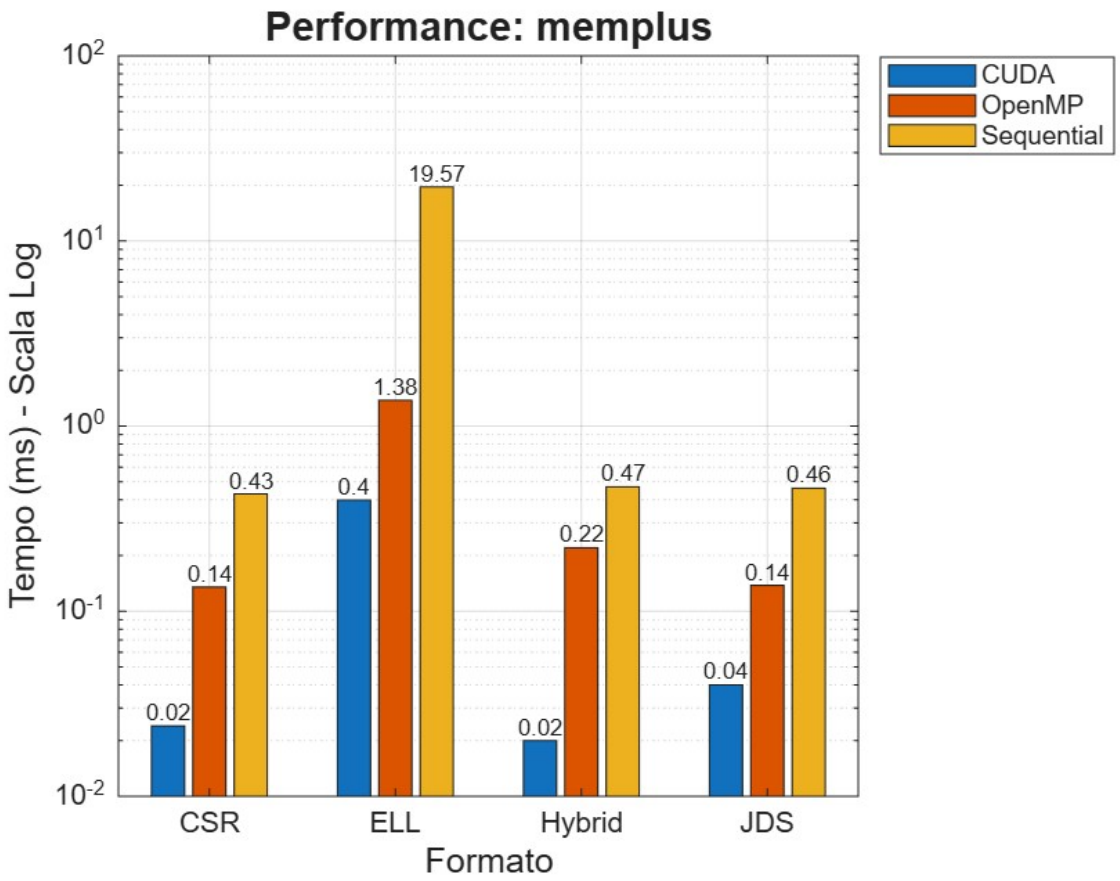

Figure 4: Matrix: `memsplus`

0.48

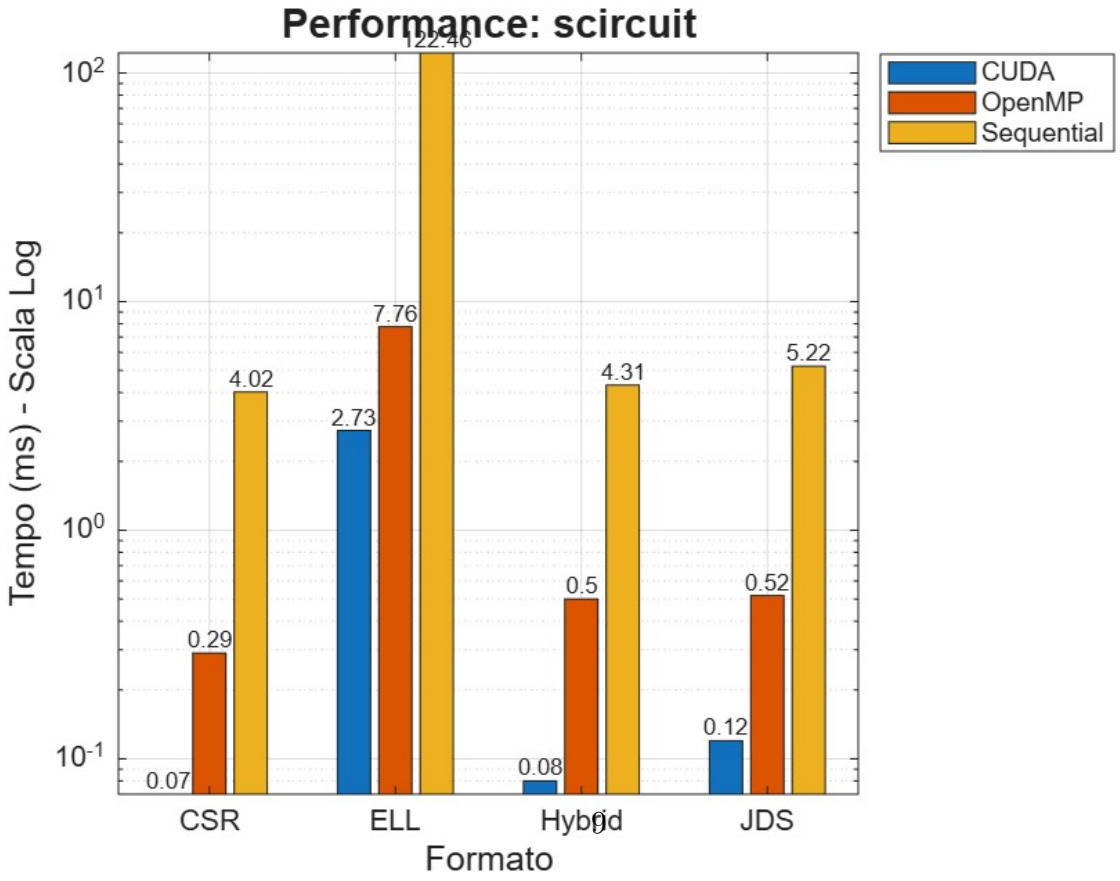

Figure 5: Matrix: `scircuit`

4.2.3 Power-Law Graphs: `web-Google`

The `web-Google` matrix represents the ultimate stress test.

- ELL Failure: The graph shows no bar for ELL. The format failed due to excessive memory requirements (> 2GB for a 60MB matrix), confirming our theoretical analysis.
- Hybrid Victory: The Hybrid format achieves the best performance on CUDA (0.93 ms), beating CSR (1.04 ms). This demonstrates that moving the few "hub" nodes (dense rows) into the COO structure allows the GPU to process the rest of the web graph very efficiently.

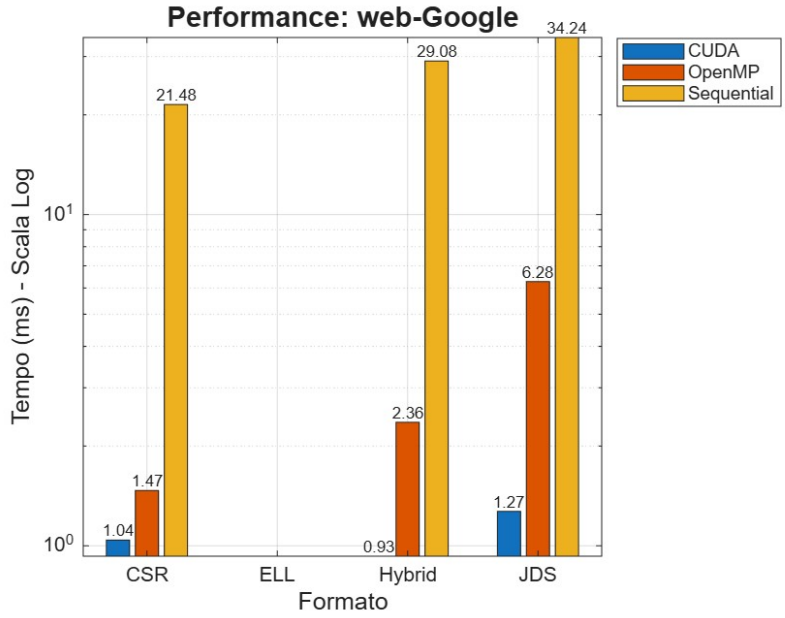

Figure 7: Performance on `web-Google`. ELL is missing (OOM). Hybrid outperforms CSR.

5 Conclusion

The project demonstrated that handling irregularity is the main challenge in SpMV on GPUs.

- ELL is ideal for structured grid problems but unstable for real-world graphs.
- Hybrid is a robust compromise that fits in memory but incurs atomic overhead.
- JDS proves to be the superior format for highly irregular (Power-Law) matrices on GPU, solving both memory coalescing and load balancing issues, at the cost of a pre-processing step (sorting rows).

While OpenMP provides a simple parallelization for CPUs, the massive bandwidth of the GPU allows for significant speedups, provided the correct storage format is chosen to match the data structure.

A Appendix: Hardware Specifications

The experiments were conducted on the following system:

Component	**Specification**
CPU	AMD Ryzen 5 3600 6-Core Processor
RAM	16,0 GB
GPU	NVIDEA GeForce RTX 3050
CUDA Cores	2560
OS	Windows 11

Table 2: System Specifications

References

[1] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th Edition, 2022. **Chapter 10: Sparse Matrix Computation**.

[2] Panruo Wu. *Lecture 13: Sparse Matrix*. COSC 4397: Parallel Computations, University of Houston, Spring 2025. Available at: `https://www2.cs.uh.edu/~panruowu/2025s_cosc4397/lec13_sparse_matrix.pdf`

[3] MadBrain. *Programming Massively Parallel Processors (CUDA Guide)*. Available at: `https://madbrain.ai/programming-massively-parallel-processors-cuda-guide-0c396732a316#808f`